

METODOLOGÍA I

Trabajo Práctico — El ciclo completo de roles (y su límite)

PARTE A: CICLO COMPLETO DE ROLES

Contexto del caso práctico:

Para este trabajo, se utilizará una función en Python encargada de validar y procesar un pago en un sistema de comercio electrónico. La función se llamará `procesar_pago`.

1. Rol Programador

Patrón que aplica: Anatomía del prompt aplicada a código nuevo.

Prompt utilizado:

"Actuá como un programador Python senior. Escribí una función con la firma exacta `def procesar_pago(monto: float, metodo: str) -> bool`. Si el monto es menor o igual a cero, la función debe levantar un `ValueError` con el mensaje exacto 'Monto inválido'. El código debe seguir el estándar PEP 8, usar type hints completos y docstrings en formato estándar."

Respuesta del modelo (resumida):

```
def procesar_pago(monto: float, metodo: str) -> bool:
    """
    Procesa un pago según el monto y método indicados.
    ...
    """
    if monto <= 0:
        raise ValueError("Monto inválido")

    # Lógica simulada de procesamiento
    metodos_validos = ["tarjeta", "efectivo", "transferencia"]
    return metodo.lower() in metodos_validos
```

Observación: Se aplica la anatomía del prompt porque no se pide un genérico "hacé un código para pagar", sino que se define explícitamente la firma exacta (nombre, parámetros y retorno), el comportamiento esperado ante la entrada inválida (el `ValueError`) y el estilo del proyecto (PEP 8 y type hints).

2. Rol Revisor

Patrón que aplica: Restricción de alcance, un único criterio de revisión a la vez.

Prompt utilizado:

"Actuá como Revisor de código. Revisá la función `procesar_pago` enfocándote ÚNICAMENTE en la seguridad. No me des sugerencias de legibilidad, formateo ni rendimiento. Comprabá si hay posibles problemas de inyección de datos o vulnerabilidades al manejar el parámetro de texto `metodo`."

Respuesta del modelo (resumida):

El modelo señala que recibir el método de pago como un string libre (`str`) expone la función a valores inesperados, inyecciones en logs o ataques si ese string va directo a una consulta de base de datos. Sugiere fuertemente utilizar un Enum de Python (ej: `class MetodoPago(Enum)`) para restringir los valores posibles desde la firma y garantizar la seguridad de los datos que ingresan.

Observación: Al dejar explícito que el modelo se limite a evaluar la seguridad y no opine sobre otros criterios, se logra una restricción de alcance efectiva. El modelo no perdió tokens analizando el estilo o la optimización matemática, sino que brindó un hallazgo crítico sobre el tipado seguro del string.

3. Rol Tester

Patrón que aplica: Exigencia explícita de cobertura, no aceptar solo el camino feliz.

Prompt utilizado:

"Actuá como Tester. Generá una batería de tests unitarios usando `pytest` para la función `procesar_pago`. Tenés que incluir estrictamente: al menos un caso de entrada típica/válida, al menos un caso de entrada inválida, y al menos un caso límite o de borde (borde numérico)."

Respuesta del modelo (resumida):

El modelo generó un archivo de tests que incluye:

1. `test_pago_valido`: Prueba con `monto=1500.0` y `metodo='tarjeta'` (espera `True`).
2. `test_pago_invalido`: Prueba con `monto=-500.0` y usa `pytest.raises(ValueError)` para comprobar la excepción.
3. `test_pago_limite`: Prueba con `monto=0.01`, que es el valor positivo más bajo posible en transacciones monetarias típicas, comprobando que se procese correctamente sin arrojar error.

Observación: Se aplicó la exigencia de casos nombrando explícitamente los tres tipos requeridos. De no haberlo hecho, el modelo suele generar solo el "camino feliz", obviando el comportamiento ante números negativos o el caso límite (0.01).

4. Debugging con cadena de razonamiento

Patrón que aplica: Cadena de razonamiento (Chain of Thought): pedir la causa raíz antes de proponer cualquier corrección.

Prompt utilizado:

"En el sistema principal, al ejecutar procesar_pago(-50.0, 'efectivo'), la aplicación completa crashea abruptamente en producción. Yo esperaba que la función manejara el error y la compra simplemente fuera rechazada. Explicame paso a paso cuál es la causa raíz de este problema ANTES de proponerme cualquier corrección en el código."

Respuesta del modelo (resumida):

- **Razonamiento previo:** El modelo explica que la causa raíz es que la función está levantando una excepción (raise ValueError) deliberadamente, lo cual detiene la ejecución normal. Si el módulo principal que llama a la función no envuelve la llamada en un bloque try-except, la excepción se propaga hasta arriba y crashea el programa entero.
- **Corrección:** Sugiere dos vías: atrapar el error en el sistema principal, o bien modificar procesar_pago para que retorne False en lugar de lanzar la excepción, en caso de que la regla de negocio sea simplemente rechazar transacciones inválidas.

Observación: Al exigir el análisis previo de la causa raíz (Chain of Thought), se evita que el modelo reescriba el código a ciegas. Primero identificó correctamente el problema de integración y luego ofreció la corrección justificada.

5. Refactorización asistida

Patrón que aplica: Restricciones explícitas de alcance + verificación.

Prompt utilizado:

"Refactorizá la función procesar_pago para extraer la validación de los datos numéricos a una función auxiliar privada. Restricciones explícitas: el comportamiento externo debe mantenerse idéntico, la firma de procesar_pago no puede cambiar, y no agregues ninguna funcionalidad nueva."

Respuesta del modelo (resumida):

```
def _validar_monto(monto: float) -> None:
    if monto <= 0:
        raise ValueError("Monto inválido")

def procesar_pago(monto: float, metodo: str) -> bool:
    _validar_monto(monto)
    metodos_validos = ["tarjeta", "efectivo", "transferencia"]
    return metodo.lower() in metodos_validos
```

Observación: Se aplicó el patrón fijando qué elementos quedaban fuera de alcance (no sumar features) y cuáles debían mantenerse (la firma). **Verificación obligatoria:** Ejecuté la batería de tests de la Ficha 3 después de aplicar este cambio y los resultados fueron exitosos (todos los tests pasaron). El comportamiento externo demostró ser idéntico.

6. Rol Arquitecto

Patrón que aplica: Role Prompting aplicado a decisiones estructurales.

Prompt utilizado:

"Actuá como Arquitecto de Software. Necesitamos escalar la lógica de los métodos de pago porque pasaremos de 3 métodos a más de 20 (criptomonedas, múltiples billeteras virtuales, integraciones bancarias). Dame 3 alternativas de diseño estructural orientadas a objetos en Python para organizar esto. Detallá las ventajas y desventajas (trade-offs) de cada alternativa, no me des una única solución."

Respuesta del modelo (resumida):

1. **Patrón Strategy (Polimorfismo):** Crear una clase base MetodoPago y subclases para cada método. *Ventaja:* Muy escalable y respeta Open/Closed Principle. *Desventaja:* Añade mucha complejidad y cantidad de archivos.
2. **Diccionario de Funciones (Callbacks):** Mapear nombres de métodos a funciones específicas. *Ventaja:* Simple y rápido de implementar en Python. *Desventaja:* Difícil de mantener si los métodos requieren gestionar sus propios estados.
3. **Patrón Factory:** Centralizar la creación del objeto de pago en una fábrica. *Ventaja:* Desacopla la creación de la ejecución. *Desventaja:* Requiere mantener actualizado el registro de la fábrica.

Observación: En lugar de aceptar una solución directa, se forzó al modelo a actuar como arquitecto entregando alternativas con trade-offs, permitiendo que la decisión final de diseño la tome el humano en base al contexto real.

7. Documentación automática

Patrón que aplica: Especificar público destinatario y nivel de detalle.

Prompt utilizado:

"Generá la documentación de la función procesar_pago para dos públicos distintos:

1. Documentación técnica para un desarrollador backend de mi equipo, detallando tipos, excepciones y casos límite numéricos.
2. Un extracto para un README público orientado a clientes de la API, explicando solo cómo usarla.

En ambos casos, evitá comentarios redundantes que solo repitan el nombre de la función (como 'esta función procesa el pago')."

Respuesta del modelo (resumida):

- **Para el Dev:** Un docstring profundo explicando que monto debe ser > 0 (alertando sobre floats muy pequeños como 0.01), detallando que lanza ValueError explícitamente y mencionando los tres strings aceptados en metodo.
- **Para el Cliente (README):** Un bloque Markdown muy simple: *"Para aprobar una transacción, llama al servicio enviando el total a cobrar y el medio elegido. Si la operación es exitosa, recibirás True."* No se mencionan excepciones ni código interno.

Observación: Se especificó el público destinatario para cada bloque, obteniendo un nivel de detalle totalmente diferente para el desarrollador (contratos técnicos) y para el consumidor externo (comportamiento general). Al darle la instrucción de evitar obviedades, la redacción final quedó mucho más profesional y útil.

PARTE B: EL EXPERIMENTO: PROMPTING SIN ESPECIFICACIÓN

Pedido deliberadamente ambiguo:

"Hacé un sistema de reservas para un restaurante."

Respuesta completa obtenida del modelo (sin editar):

¡Claro! Aquí tienes un ejemplo básico de un sistema de reservas para un restaurante escrito en Python. Este sistema permite agregar reservas y ver la lista de reservas actuales.

```
class SistemaReservas:
    def __init__(self):
        self.reservas = []

    def hacer_reserva(self, nombre, fecha, hora, cantidad_personas):
        reserva = {
            'nombre': nombre,
            'fecha': fecha,
            'hora': hora,
            'personas': cantidad_personas
        }
        self.reservas.append(reserva)
        print(f"✅ Reserva confirmada: {nombre} - {fecha} a las {hora} ({cantidad_personas} personas)")

    def mostrar_reservas(self):
        if not self.reservas:
            print("No hay reservas en el sistema.")
        else:
            print("\n--- Lista de Reservas ---")
            for r in self.reservas:
                print(f"{r['fecha']} {r['hora']} | {r['nombre']} - {r['personas']} personas")

# Ejemplo de uso:
mi_restaurante = SistemaReservas()
mi_restaurante.hacer_reserva("Lautaro", "2026-10-25", "21:30", 4)
mi_restaurante.hacer_reserva("Julietta", "2026-10-25", "22:00", 2)
mi_restaurante.mostrar_reservas()
```

Este es un script sencillo. Si quieres llevarlo a un entorno real, deberías conectarlo a una base de datos (como MySQL o SQLite) y agregar una interfaz gráfica o web.

Reflexión final (Análisis del experimento):

Al pasarle el prompt completamente vacío de contexto, el modelo tomó por su cuenta múltiples decisiones arquitectónicas y de negocio que en la Parte A habíamos controlado de forma rigurosa. Decidió unilateralmente el formato (una clase orientada a objetos que guarda datos en memoria RAM temporal), el alcance (solo permite agregar y listar reservas, ignorando por completo la necesidad lógica de cancelar o verificar si hay mesas disponibles) y los criterios de éxito (asumió que imprimir un mensaje en la consola equivale a una transacción exitosa). Tampoco contempló ningún caso de borde: el código acepta números negativos para la cantidad de personas o textos sin sentido en lugar de fechas válidas.

El problema que observamos acá no se resume a un "prompt mal escrito" o demasiado corto. El problema de fondo es que delegamos una tarea compleja a un sistema probabilístico sin establecer un objetivo verificable. El modelo no tiene forma de saber cuándo su trabajo terminó de manera correcta porque las reglas del negocio jamás existieron en su contexto; simplemente encadenó los tokens estadísticamente más probables asociados a las palabras "sistema", "reservas" y "código". Esto demuestra que un sistema autónomo operando en varios pasos requiere obligatoriamente restricciones claras, contratos (como tests o validaciones) y un "estado de éxito" medible. Sin esas especificaciones previas, el modelo se ve forzado a alucinar sus propios requerimientos, entregando una solución de juguete inservible para producción.